Common Problems

Early Access

Harmful Content

By Stefan Mai · Published Apr 28, 2025 · medium

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

▷ Watch Now

Understanding the Problem

🔍 **What is Facebook?**

Facebook is a social network centered around "posts" (messages). Users consume posts via a timeline composed of posts from users they follow or more recently, that the algorithm predicts they will enjoy. Posts can be replied, "liked", or "shared" (sometimes with commentary).

On a social network, people are free to express themselves via posts, comments, and other forms of content. Unfortunately, some of this content is harmful or offensive. While there exist social networks that are entirely open, most social networks moderate harmful content to comply with the law and ensure a positive experience for their users.

For this problem we're going to focus on Facebook's post moderation system.

Problem Framing

We'll start by establishing a framing for the problem we're looking to solve. We want to clarify the problem and the constraints we're working under.

Clarify the Problem

To start, we're going to ask the interviewer some questions that will help us clarify how the system will operate. There's a bit of an art to this: you want to be specific enough to get a clear

picture of the problem, but you want to be efficient with your time and questions you ask your interviewer. For more senior candidates, making assumptions (and noting those assumptions you made) is a good way to show your independence for the interviewer.

Some example questions and interviewer answers (they might just ask you to make assumptions):

- Are posts text-only or can they include images, videos, or other media?
  - *Posts can include text, images, and videos, but let's focus on just images and text for this interview to keep things simple.*
- What kind of content do we want to detect?
  - *We want to detect all harmful content: nudity, violence, all the way to terrorism and human trafficking.*
- What happens if we find harmful content?
  - *Let's assume we can automatically remove content we're confident is harmful and demote likely harmful content. We also have a small team of moderators who can review content, but they will have limited capacity.*
- What are the consequences of false positives and false negatives?
  - *We want to make sure people are free to express themselves, so we don't want to remove anything we aren't 95% confident is harmful. Users who have their content removed incorrectly will be unhappy and may leave the platform. On the other hand, users who are exposed to excessive harmful content may leave the platform if it hampers their experience.*
- How quickly do we need to detect harmful content?
  - *Up to you! We want to minimize the harm caused.*
- Do we have data available for the initial training or will we need to cold start?
  - *You can assume we have a small amount of labelled data (50k examples, 50% harmful and 50% benign) available for the initial training.*
- Do we need to find harmful comments, or just posts?
  - *You're free to use comments, but we're only interested in posts.*
- How many posts are made per day?
  - *Let's assume 1B posts per day.*

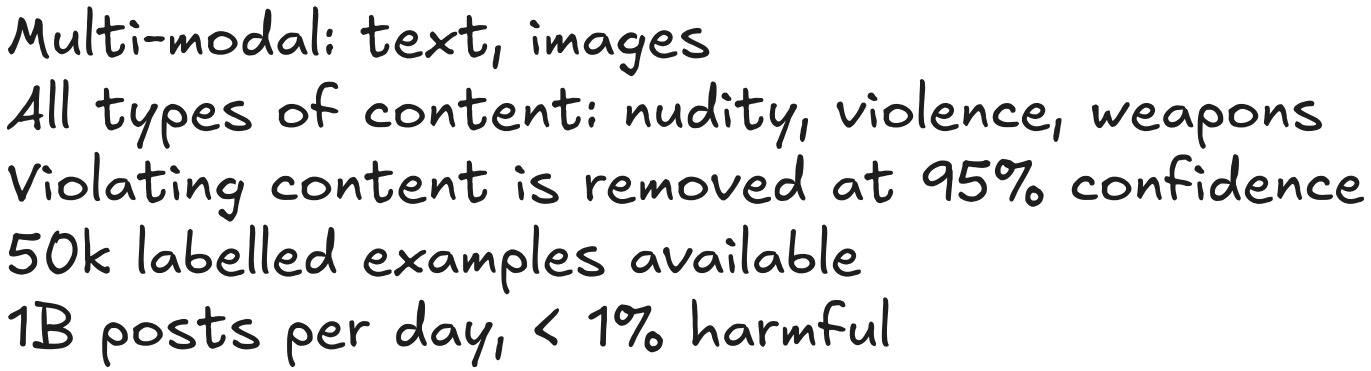
- How common are harmful posts?
    - *Relatively rare, < 1% of posts are harmful. But with billions of posts per day this can add up.*

Some things that we might realize here:

- We've established this is a multi-modal content classification problem.
- 95% precision is a threshold for automated deletion.
- We have a good idea of the scale of the problem and what type of data we're dealing with.

This should be enough to get us started!

We'll be taking notes on our whiteboard as we go along, but this is just to keep on the same page as our interviewer and not judged as a final product:



Problem Clarification

Establish a Business Objective

Next, we need to translate our problem into a business objective. The business objective is distinct from our loss function or ML objective: it's the end-goal the business cares about. We can think of much applied ML engineering as an optimization problem over a business objective.

For many teams they'll spend years optimizing the same objective and will have put a deep amount of thought into its construction. Strong candidates are able to think deeply about the problem and come up with a business objective that is both aligned with the business's goals and a good proxy for the ML problem.

Bad Solution: Maximize the amount of harmful content removed

Bad Solution: Maximize the accuracy of our model

Good Solution: Maximize the amount of content removed, subject to precision guardrails

Great Solution: Minimize the number of successful user reports of harmful content

Great Solution: Minimize the number of views of harmful content, subject to precision guardrails

Let's move forward with the last one: minimize the number of views of harmful content, subject to precision guardrails.

> By moving our business objective to a higher level of abstraction, we're able to answer some questions that otherwise would be hard to answer. For example, "do we need to classify the content immediately or can we wait?" has an objective standard now: the longer we wait, the more views against potentially harmful content we're going to accumulate.
>
> There's also a deep insight here. Harmful content becomes easier to detect the more people are exposed to it. Their behaviors (whether blocking, unliking, commenting, etc.) make the challenge of classification of any particular piece of content easier with time. We can exploit this in our solution!

Decide on an ML Objective

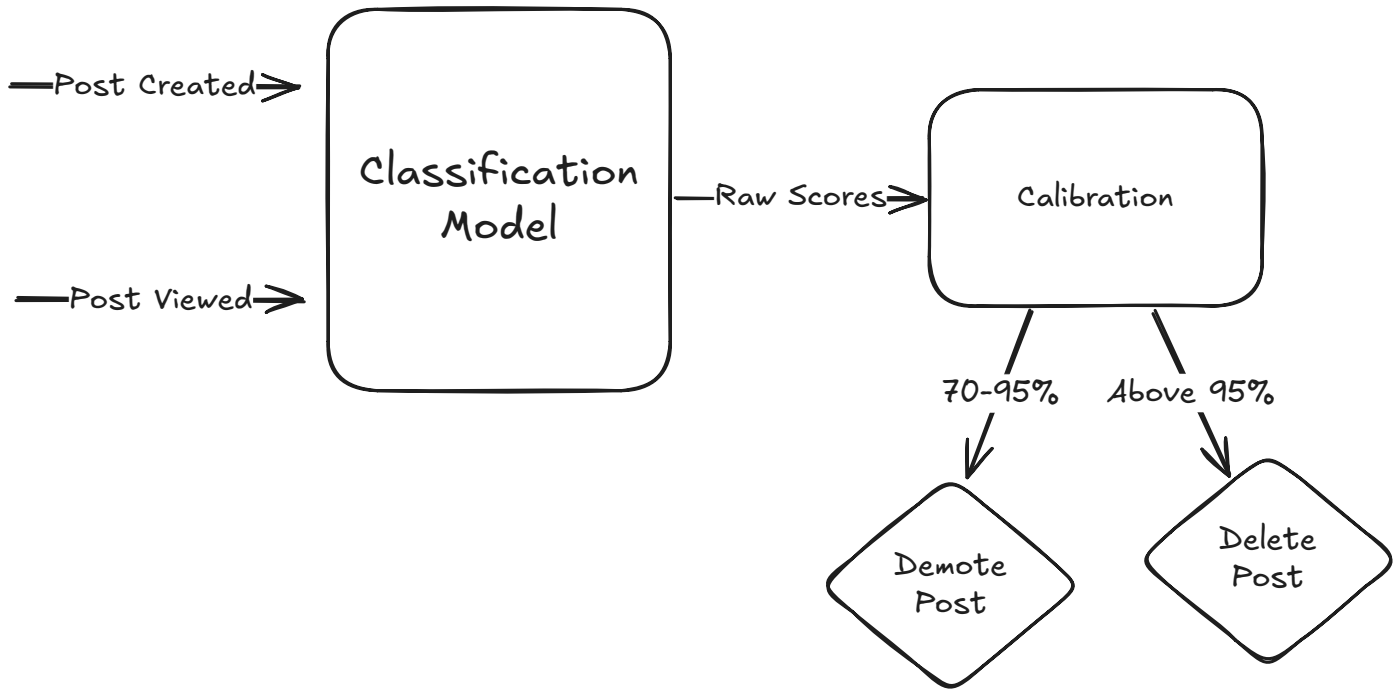
With our business objective in mind, we can now decide on an ML objective. For this problem, it's obvious that we're looking for a binary classification: is this post harmful or not? We'll defer making decisions on the loss function until a bit later.

High Level Design

Our high-level design for this problem will start with a simple classification pipeline. When posts are created (or, sometimes, when they are updated with new behavioral data), we'll classify them as a harmful or not. Our classifier will need to be calibrated by a layer which guarantees we meet our precision guardrails. Finally, we'll need to feed the output of our classification to a system which can take action to either delete the post, flag it for review, or some other action.

We can write this on the whiteboard as a communication aid, but we don't have to. The important thing is the interviewer is aware of the pieces involved and how they relate to one another.

> Some candidates over-optimize for their whiteboard presentation at the expense of both their mental bandwidth and the interviewer's time — this is not a good tradeoff!
>
> Using the whiteboard can be valuable as a communication aid, but don't stress over arrows and lines. Your job is just to get on the same page and minimize miscommunication.



Basic High Level Design

Data and Features

Next we'll move to discuss the data and features we'll use to train our model. Data discussions can be some of the most time-consuming parts of an ML design interview so it's important to be strategic about what we discuss here.

> For most problems, there is a theoretically infinite amount of data we can use to train our model. The challenge is that most of this data isn't actually that useful for the problem at hand. Senior+ candidates need to demonstrate they are capable of generating hypotheses about what data is actually useful and showing how they might prioritize the evaluation. It's less important that you're right about any given dataset, and more important that you're able to justify your choices.

Training Data

There's three categories we might consider for training data:

Supervised Data

While we were clarifying requirements our interviewer mentioned we have a small amount of labelled data (50k examples, 50% harmful and 50% benign) available for the initial training. This is our most valuable dataset and we should use it!

There may be additional data available to use. For instance, there are plenty of NSFW (not safe for work) datasets available online. We may be able to use these to augment our training data, but it won't have any platform-specific features (more on this in a second) and we'll need to be careful that this doesn't introduce bias into our model.

We should also ask our interviewer if we'll have access to fresh labelled examples. There's a potential discussion here around data drift and how we'll evolve our model as the distribution of harmful content evolves. It's also important that we're providing the model "hard" cases that are prone to false negatives or false positives. Obviously harmful or benign content will be significantly less valuable to our model.

Data drift is a common cause of model performance degradation. If your interviewer asks you a question that sounds like "your model tested well but performance has gradually worsened over time", there's a good chance the root cause is drift!

Semi-Supervised Data

Beyond labelled data, we earlier talked about the possibility of using user reports as a business objective. These user reports also make a decent proxy label that is likely correlated with the true label.

We would expect that the volume of user reports would be significantly higher than the amount of labelled data we have (say 10M reports vs 50k labelled examples) which will be important for any larger models we wish to train.

Self-Supervised Data

One final thing we might note here is that user comments are a rich source of information. "Gross!", "I didn't want to see that", "This is disgusting" are all potentially useful signals for detecting harmful content.

It's hard to expect "gross!" to *always* be a good signal for harmful content (it probably appears in many other contexts too), but a model trained to predict comments from the post body would learn substantially more useful representations for posts than a model trained on post bodies alone. And we'd expect to have orders of magnitude more posts with comments than we have of either labelled examples or semi-supervised data via user reports. This is a great way to get more information out of our data!

At Facebook, early image models were trained to predict tags on Instagram images. The resulting embeddings were surprisingly effective at capturing the semantics of the image and used across the company.

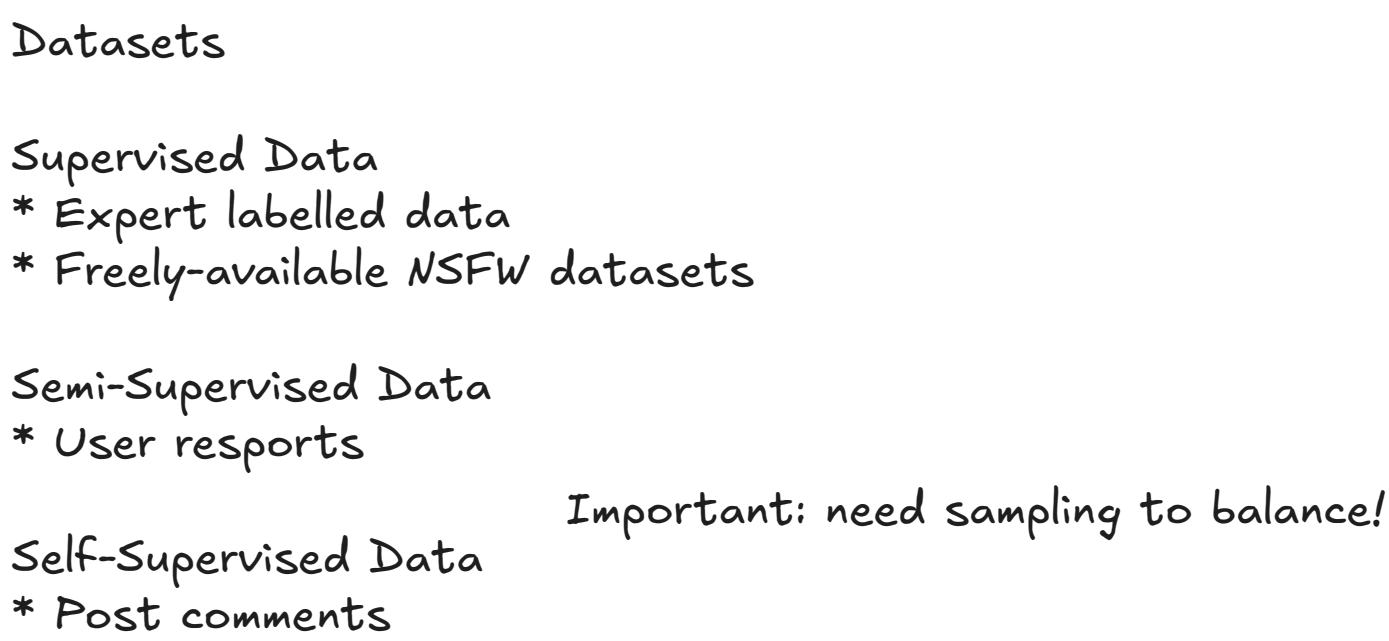
Class Imbalance

To round out our discussion of data, we need to address the elephant in the room: class imbalance. We mentioned earlier that harmful content is relatively rare (<1% of posts). This presents a significant challenge for our training process and experienced ML engineers will be quick to point this out.

If we were to train on the raw distribution of data, our model would likely learn to predict "not harmful" for everything and still achieve 99% accuracy!

For our solution, we'll use a combination of balanced sampling during training and loss weighting to fine-tune the precision-recall tradeoff, but for now we'll acknowledge this as an important consideration and earmark it for later discussion.

Our whiteboard is just for keeping notes at this stage and might look like this:



Datasets

Features

With our datasets established, we'll need to think about what features we can use to train our model. Social networks are **rich** sources of information, which makes enumerating possible features a daunting task, especially given the time limitations. In order to move forward, we'll create some crude categorizations based on hypotheses about what might be useful.

Feature discussions are easily the biggest tarpit for candidates because there's just so much to talk about. Maintaining focus on the most impactful features, elaborating sufficiently that your interviewer sees your intent, and keeping moving are essential to a successful feature discussion.

Content Features

The features we expect to be the **most** informative are those directly related to the content of the policies: the text and image contents of the post. These necessarily are first-class features for our model and might be treated separately from the rest of the features.

While we could model aspects of the post distinctly (e.g. hashtags, topics, etc.) we're going to keep things high-level because we expect to use a powerful model that can learn these features implicitly.

So our two inputs are:

- The concatenated text input including the post body, hashtags, etc.
- A collection of images attached to the post

Now we have a **multi-modal** classification problem. If we didn't mention this earlier, now's a good time to mention it to your interviewer.

Behavioral Features

As we talked about earlier, we expect that *how users respond* to posts might be just as valuable as the content of the post itself. There are a number of angles we can look at behaviors:

- How many comments that signal harmful content were there?
- How many negative reactions (angry reactions, hides, etc.) have been made on the post?
- What's the ratio of shares to views?

Behavioral features are fast-moving and subject to change, so modelling them as simple numeric signals is a good way to keep things simple. Each of these numeric features will need to be normalized and encoded to be useful for the model and will need to be corrected by the

age of the post. We would expect that all posts at `T=0` have 0 negative behaviors associated with them but that doesn't necessarily mean they're benign.

A simple solution is to use ratio like `negative_reactions_per_view` or `shares_per_view` which will naturally correct for the age of the post. However, we need to be careful with posts that have very few views, as the ratios could be unstable or undefined. We can address this by using Bayesian averaging that incorporates a prior belief about the expected ratio. This ensures our features remain stable even when the denominator is very small or zero and distinguishes posts that may have had an "unlucky" first few views from those that have consistently received negative signals.

This discussion introduces an important fork for our design: if we use behavioral features we need to consider classification **not only when the post is created, but also later** as new behavioral inputs arrive.

This means we have some new problems to consider: how do we include these features in our training set? When do we trigger (re-)classification of the model? etc. We'll make a note of these to our interviewer to acknowledge them and defer the discussion for later when we discuss the inference process.

> ML design interviews are chock full of potential rabbitholes. Some of them are productive and demonstrate depth to your interviewer, others tear you off the main thread and make it hard to complete the task.
>
> **Earmarking** is a useful technique for managing this tradeoff — you make a verbal note (and perhaps write it on the whiteboard) to your interviewer that you'd expect to cover a specific topic later. You may not get the time, but by showing your interviewer that you're thinking about the implications, you're preventing a red-flag from being raised that you "missed it" without necessarily burning the time to completely resolve.
>
> If your interviewer thinks it's important to go into, they'll probe with additional questions on the spot and you'll know it's a good place to dive in.

Creator Features

What the post contains and how the audience interacted with it will both be important signals, but we should expect that some users are more/less likely to **author** harmful content. But how can we represent this information?

There are two complementary approaches to realizing this:

User Embeddings

First, we want the full richness of the user's profile to be represented in the model. If a grandma is always posting in a fuzzy cat group she is probably less likely to post harmful content than the teenager who frequently posts in the "death videos" group.

One way for us to incorporate this information is to create an embedding of the user's profile and use that as a feature. So long as the embedding incorporates information relevant to the task of identifying harmful content, we can use it to make predictions. We'll talk more about training these embeddings in deep dive discussions.

User Contextual Features

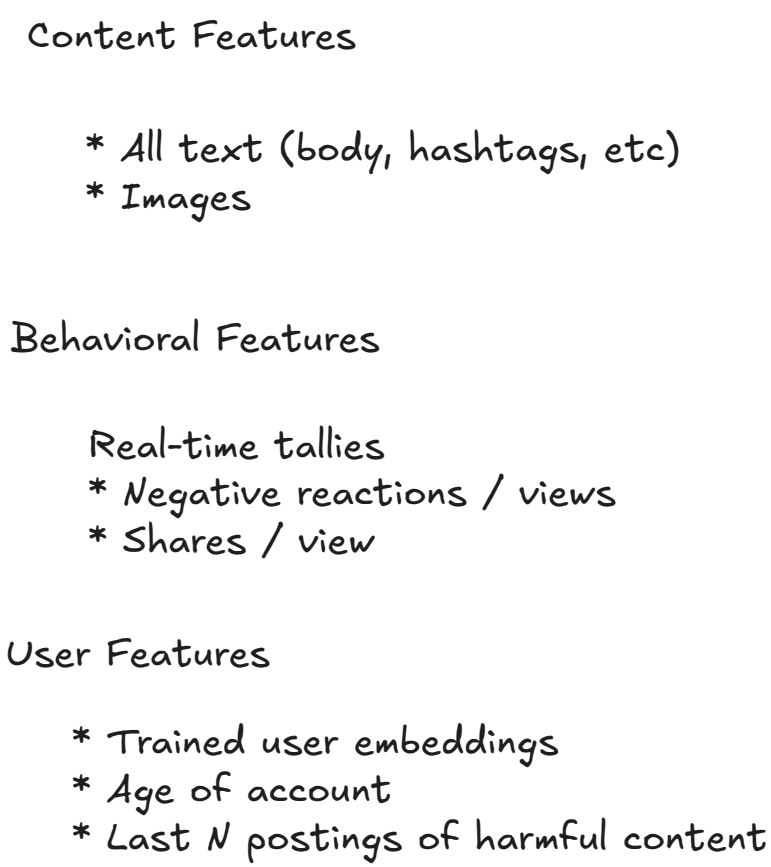
But where embeddings can be powerful, they tend to be better at expressing averages than individual cases. If grandma suddenly goes on a rampage of posting harmful content, we'd like to be able to account for that.

The most simple way for us to do this is to record real-time tallies of important signals. For instance, we might count the number of reports, dislikes, or shares for each user. Like prior behavioral features, these need to be normalized to correct for the age of the user and the fact that some users are more active than others.

> Hacked accounts and bots are also common sources of harmful content. We can use the same real-time tallies to provide those inputs to our model by adding features like `account_age` or `number_of_login_countries` to the model.

The combination of these two approaches gives us a reasonably comprehensive view of the user input: slow-but-rich embeddings that represent average historical activity, combined with fast-but-noisy real-time signals and tallies.

Our whiteboard might look like this:



Features Whiteboard

Modelling

Ok we've got an objective, we have data, and we've got features. Let's talk models.

Benchmark Models

We'll want to start our model discussion by thinking about a simple baseline we might employ for this problem. In this case, a basic logistic regression model with our numeric and embedding features would probably make a good start. A logistic regression *won't* implicitly model dependencies between features, so there's going to be a lot of potentially harmful content that it fails to detect. But it's lightning fast, explainable, and easily updatable to give us a good starting point to talk about tradeoffs.

Starting with a benchmark model helps to temper perceptions of overcomplicating your solution. It also shows maturity as an engineer: sometimes a simple solution is all you

> need!

Model Selection

When selecting our model architecture, we need to consider several key requirements:

1. The ability to handle multiple modalities (text and images)
2. The capacity to incorporate behavioral and user features
3. Efficient inference to handle 1B posts per day
4. The ability to update predictions as new behavioral data arrives (and handle missing data when it hasn't arrived yet)

Let's look at some potential approaches:

Bad Solution: Independent Unimodal Models

Good Solution: Late Fusion Model

Great Solution: Multi-Modal Transformers

For our solution, we'll use a multi-modal transformer architecture. This gives us the best shot at catching subtle forms of harmful content and we'll address the performance challenges separately.

Model Architecture

Our model architecture consists of three main components:

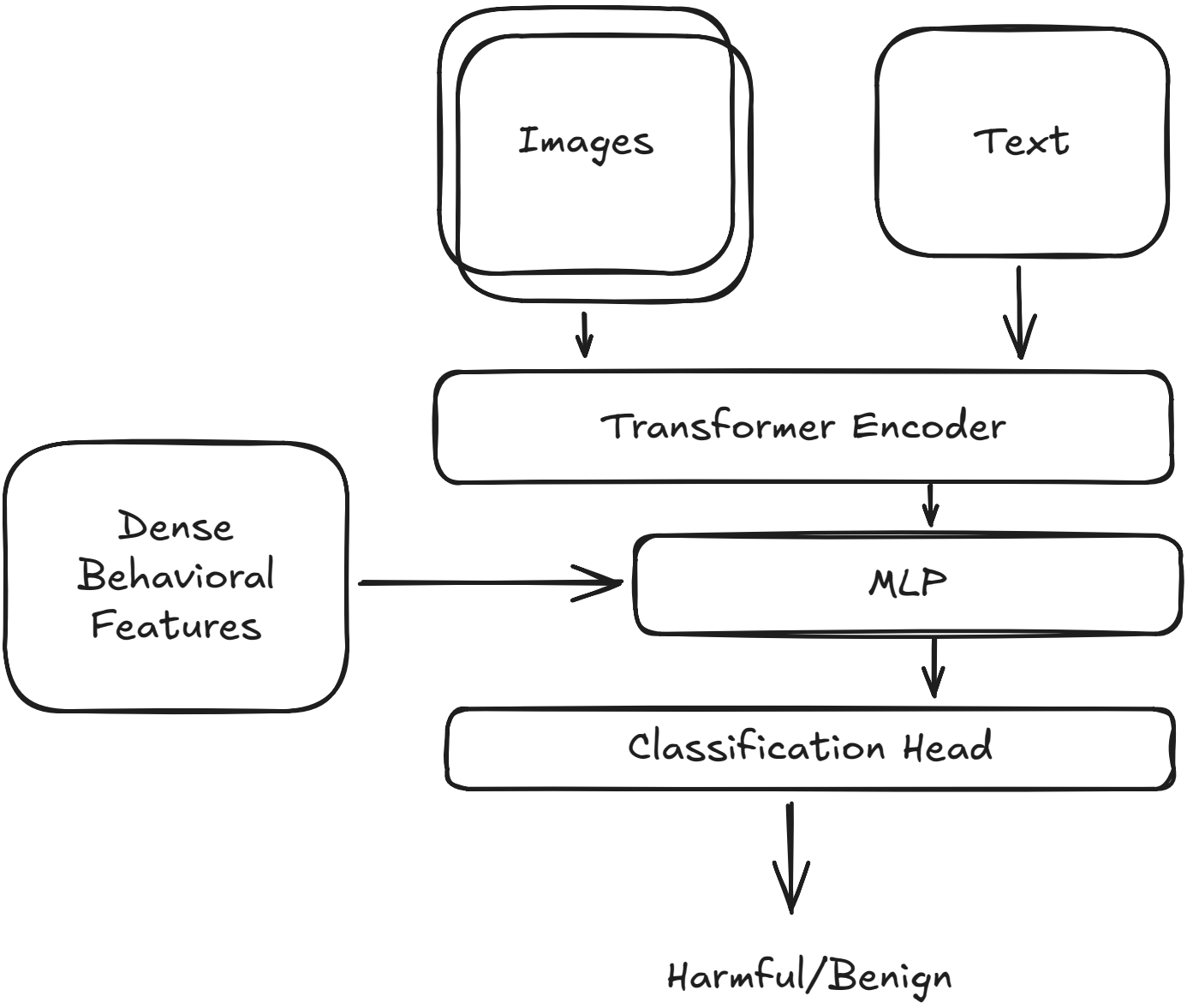
1. **Multi-modal Encoder**:
    - A vision transformer (ViT) for processing image patches
    - A text tokenizer and encoder for processing the post body, hashtags, etc.
    - Cross-attention layers to combine modalities
    - Additional embedding layers for behavioral and user features

2. **Lightweight Update Network**:

- A small network that can efficiently merge dense, behavioral inputs with the heavy content-specific embeddings
- Takes the original model's embedding and new behavioral features as input

3. **Classification Heads**:

    - Multiple classification heads for different tasks we'll engineer (talk about this in a second)
    - Shared representation layers to leverage common patterns
    - Calibration layer to ensure precision requirements are met



Model Architecture

Loss Function

Our loss function needs to balance two main objectives:

1. **Classification Performance**: Binary cross-entropy loss for the main classification task which is used to discriminate between harmful and benign content.
2. **View-weighted Loss**: Our business objective is focused on content that gets views, not all content. We'll weight the loss by the potential view count to align with our business objective and focus on content that is more likely to be harmful *and* viewed.

> While it might be tempting to simply weight by the raw views of the content, this is not a good idea. Views on social networks are power-law distributed so you can have some content with hundreds of millions of views whereas other pieces get 0 views. This means the loss function will be dominated by the high-view content, leading to instability and a hard time learning the important patterns in the data.

Instead, we can add a small logarithmic term to the loss function that penalizes the model for missing high-view content. This is a simple and effective way to ensure the model pays attention to content that is more likely to be harmful *and* viewed.

> There's a feedback loop here: if our production model is removing content before it gets lots of views, we're going to bias the view weighting in our training data. We'll need to account for this by imputing the predicted views (likely requiring some sort of holdout).

Multi-Task Loss

Earlier we talked about the value of using user reports as a source of semi-supervision. We can incorporate this into our training by using a multi-task learning approach.

We'll train two separate heads, but note that we can engineer additional tasks as the system evolves:

1. **Primary Task**: Binary classification of harmful content with view-weighted BCE loss

```
L_primary = BCE(y_true, y_pred) * min(log(1+views), c)
```

2. **Report Prediction**: Predict the likelihood of user reports

```
L_reports = MSE(report_rate, predicted_report_rate)
```

The final loss is a weighted sum of these components:

```
L = α * L_primary + β * L_reports
```

where α and β are hyperparameters we can tune where we'll prefer the primary task by keeping α high.

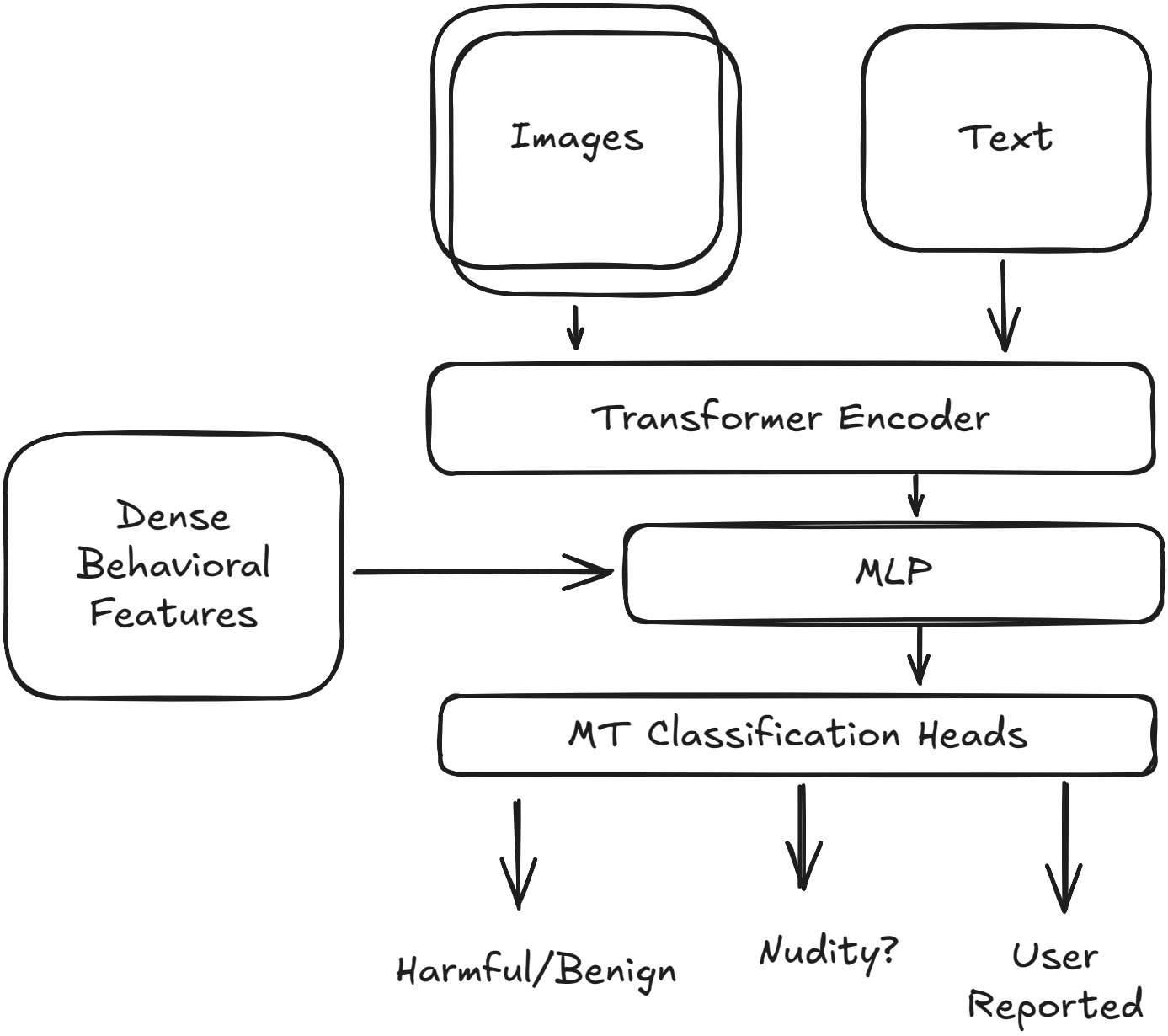
When it comes to predictions from our production system, we'll ignore all heads except for our primary task.

This multi-task approach has several benefits:

1. It helps the model learn robust representations by forcing it to predict related tasks
2. It provides a way to leverage our semi-supervised data effectively
3. It gives us more diagnostics and a way to see how the model is "thinking"

> Many candidates will opt to include a discussion about multi-class learning here where we try to predict the "type" of harmful content: nudity, violence, etc.
>
> The typical way to do this is to apply a softmax to assign probabilities. We *don't* want to do this here because harmful content can easily have multiple classes simultaneously.
>
> A more correct formulation would be a multi-label classification. Having additional secondary classification heads for each label would give us a way to distinguish the different types of content and some additional diagnostic/explanatory value, provided our labels are accurate.

Our final diagram might look like this:



Multi-Task Architecture

Inference and Evaluation

Inference System

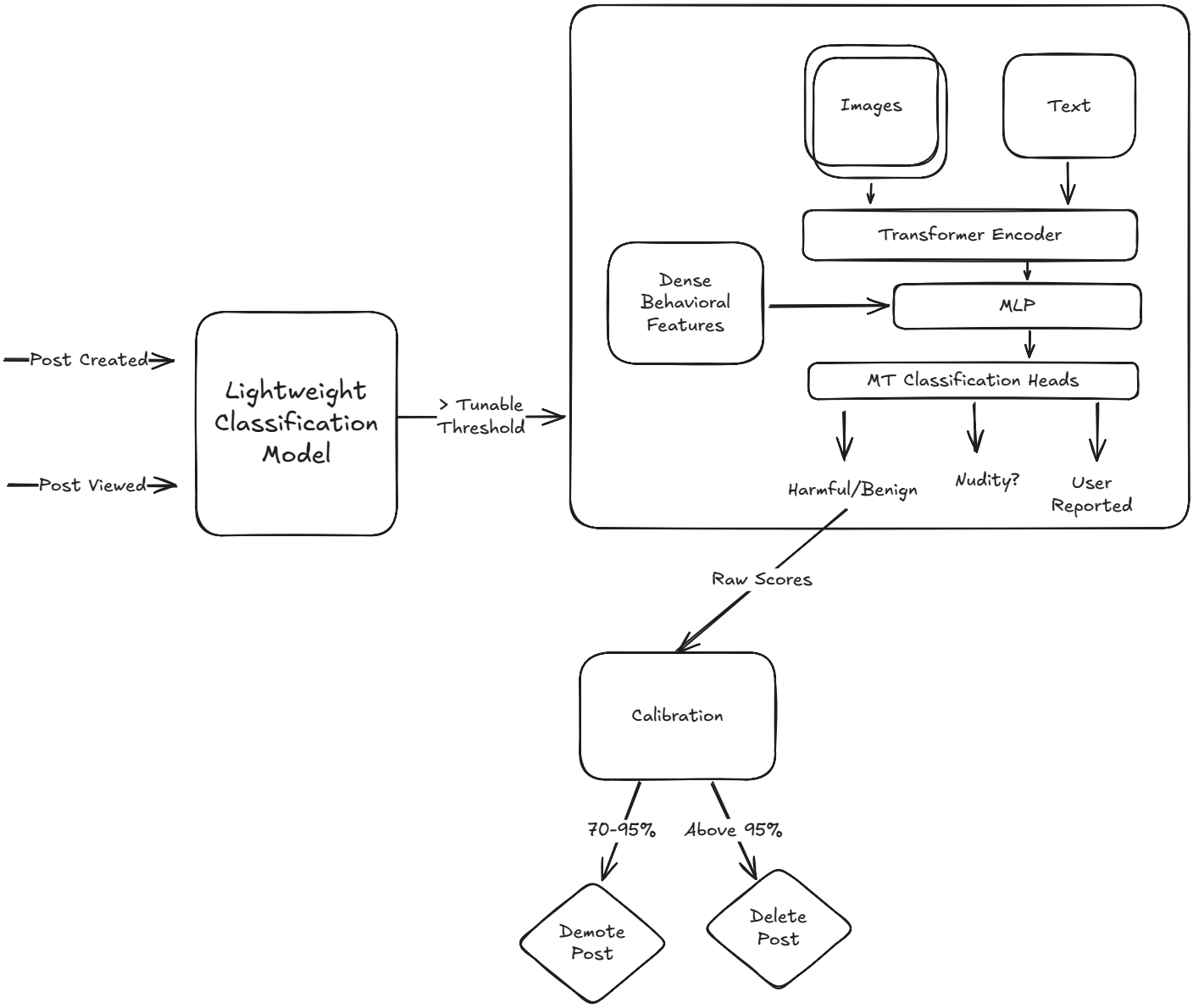
Bringing this model into production requires us to contend with two major challenges: scaling the compute necessary for 1B posts/day and triggering the model at the appropriate time. Let's talk about them both.

Scaling

Our multi-modal transformer is hefty and expensive, likely requiring expensive GPUs to run efficiently. To reduce the compute footprint, we can implement several optimization strategies. We can make the model as efficient as possible by using **Quantization-Aware Training** to quantize the model and reduce the memory required for each parameter. This approach maintains accuracy while significantly decreasing computational demands.

We can further optimize by employing caching for our encoders. Since a non-trivial number of posts will use identical images or text, we can avoid redundant forward passes by storing and reusing these encodings. This caching strategy provides substantial performance gains, especially for viral content or common phrases.

Another effective approach involves implementing a two-stage architecture for this problem. Rather than running our expensive model on all content, we can create a lightweight, distilled model using fewer features, trained in a teacher/student fashion to approximate the larger, more expensive model. This lightweight model serves as a filter to quickly identify obvious non-violations, allowing us to reserve the computational resources of our heavier model for only the content that requires deeper analysis. This cascading approach dramatically reduces overall computational requirements while maintaining detection quality. We also get a nice hyperparameter to dial up/down the resource consumption of our system!



Lightweight Model

Triggering

Our inference system needs to handle both initial classification and updates based on behavioral signals. Whenever a new post is created we'll run it through our lightweight model. If the score is high enough, we'll pass it to our heavy model for a final classification.

We also want to be able to re-trigger the model when important events happen: user reports, a significant number of views, disappointed comments, etc. Our two-staged approach with caching is well-suited for this because we can quickly re-classify content that has been updated and we don't need to recompute the parts of the model that haven't changed (i.e. the image and text encoders).

When re-triggering the model, we need to be careful about how we represent this in our dataset.

- **Positive Suppression**: If few positive examples are available with behavioral data (because they're removed by our system), our model will learn a false conclusion: that *any* behavioral indicators are a sign that the content is benign.
- **Calibration**: Calibration is also trickier. We need to ensure our calibration layer is robust to the fact that we're re-triggering the model on the same content multiple times.

Evaluation Framework

Once things are ready to go, we need to talk about evaluation. Our evaluation strategy has two imperatives: in online experiments, we want to prove that a new model is actually better at our business objective of reducing views of harmful content (subject to our guardrail). For our offline evaluations, we want statistics that are as predictive as possible towards these online results.

Online Evaluation

For our online evaluation, we'll run two models side-by-side (candidate and control) and take action only using one or the other. We need to compare their performance. This *necessarily* involves additional labelling because we expect the two models to flag different regions of the content distribution.

If we were to randomly sample examples, we'd need a lot of samples in order to measure a prevalence below 1% with any precision. Instead, we can do **Importance Sampling** of impressions using the scores from each model.

Intuitively: we'd expect that most content scored .99 by either model is likely a positive, and that scored 0.0 is likely negative — so we don't need as many labels from these scores in order to get a good estimate. We can down-sample them for labelling and reweight them to remain unbiased for a cheap variance reduction.

We can also use some of our proxies like user reports, appeals, etc. to get a sense of the performance differences of the models.

Offline Evaluation

For offline evaluation, we'll run the models against our test sets and compare their performance. Good metrics to consider here are PR-AUC and Recall@Precision95: the former is a good, stable indicator of the overall strength of the model, whereas the latter is more closely aligned with the action threshold of our model (but with higher variance since it sits on a single point on the PR curve). We can also use an impression-weighted variant of these metrics to align with our business objective.

Some interviewers might be interested in us establishing that our model is fair to all users. We can partly cover this by evaluating the model's performance across different user groups to look for significant discrepancies. A more nuanced assessment of model bias and fairness will require a significantly deeper discussion!

Deep Dives

In Deep Dives, we'll return to some of the topics we've deferred to discuss OR our interviewer might want to dive deeper into a specific topic. We left one topic out from our initial discussion: how we're going to train our user embeddings. Let's talk about that now.

Embedding Models for Users

We talked earlier about using user embeddings to represent the user's profile. Our interviewer might want to know *how* we're going to train these embeddings and there's plenty of options for us to talk about here.

> For Facebook, generally accessible embedding features like `world2vec` (a play on `word2vec`) provide a drop-in solution for this problem (and also a hint that there's a lot of utility here!).

Bad Solution: Simple Embedding Model

Good Solution: Transductive Graph Embedding Models

Good Solution: Inductive Graph Embedding Models

What is Expected at Each Level?

For this problem, mid-level engineers are going to be expected to demonstrate practical proficiency and a modest level of depth. They'll need to be able to frame the problem, decide on the most informative features, work through modelling tradeoffs, and speak to some of the implications in production. Generally speaking mid-level engineers will differentiate themselves by showcasing their ability to deliver a credibly-workable solution.

Senior-level engineers will take this a step deeper. Their expertise in feature engineering will be apparent from their choices of features to decisions on how to encode and represent. They'll have a broader toolkit to pull from in terms of modelling choices and will recognize many of the difficulties systems like this face in production (feedback loops, data drift, etc.).

Staff-level candidates will be expected to breeze through the straightforward aspects of the problem to focus their efforts on the most interesting/impactful areas. They'll have a solid understanding of some of the research in multi-modal models (even if it's not their specialty) and will be able to speak to the tradeoffs between different approaches. They'll volunteer many of the challenges that may be expected and will proactively lead deep dive discussions. Staff-level candidates usually also recognize the bottleneck of labelled data and often propose creative solutions to augment the data available to them or optimize the way it's collected.